

Web 3層アプリケーション

この回では、第3回以降の Azure ハンズオンで登場する VM、コンテナ、DB などを「Web 3層のどの役割か」という共通の見方で整理します。

オンライン座学

50分レクチャー + 10分 QA

2026年6月25日

役割で Web アプリを見る

今日は個別サービス名を暗記するより、Web/AP/DB の責務、処理の流れ、以降の演習で作る範囲を説明できる状態を目指します。

GOAL 1

処理の流れ

ブラウザー、Web サーバ、アプリケーション、データベースの連携を追える。

GOAL 2

3つの層

Web 層、AP 層、DB 層がそれぞれ何を担当するか説明できる。

GOAL 3

構成の違い

モノリシック構成と Web 3層構成の違いを説明できる。

GOAL 4

次回への接続

第3回から第6回で作るものを、3層の観点で見通せる。

第3回から第6回で作るもの

これからのハンズオンは、VM 上の NGINX から始め、コンテナ、Container Apps、PostgreSQL を組み合わせる Todo アプリへ段階的に広げます。

第2回で地図を持ち、第3回から実装で確かめる



以降の Portal 操作で「何を作っているか」を判断するための共通言語にします。

なぜ先に Web 3層を学ぶのか

Azure Portal の操作手順だけを追うとリソース名が点で残るため、先にアーキテクチャの地図を持ち、作成する対象の意味と接続関係を確認します。

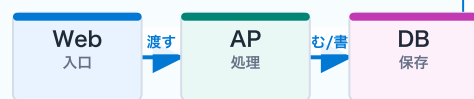
手順の点を、役割とつながりの地図に変える

手順だけを追う



関係が見えず、名前だけが残りやすい

構成を見ながら操作する



役割と接続関係で意味を確認できる

Web アプリケーションの基本

Web アプリケーションは、ブラウザからの要求を受け取り、サーバー側の処理を経て、HTML や JSON などの結果を返す一連の仕組みです。

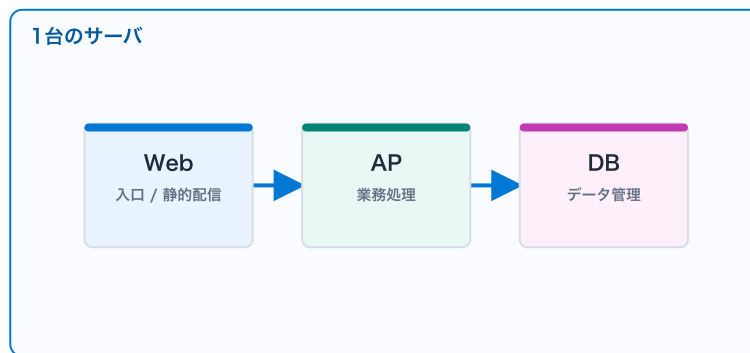
リクエストを受け取り、処理して、レスポンスを返す



1台ですべて動かす構成

小規模な構成では Web、アプリケーション、DB を1台に同居させることもあり、まずはこの単純な形から役割の違いを見ていきます。

1台の中に役割が同居する構成



始めやすい

分けて拡張しにくい

モノリシック構成の利点と注意点

1台にまとめた構成は始めやすい一方で、規模が大きくなると拡張、障害対応、変更範囲の分離が難しくなるため、利点と注意点をセットで確認します。

観 点	利点	注意点
構 築	必要なサーバやネットワーク構成が少ない。	役割が同居するため、後から責任範囲を分けにくい。
通 信	同じサーバ内で処理できるため、経路が単純。	障害が起きたとき、影響範囲が Web/AP/DB 全体へ広がりやすい。
運 用	見る対象が少なく、初期学習で全体を追いやすい。	規模が大きくなると、一部だけを拡張・変更しにくい。

Web 3層アーキテクチャ

Web 3層では、利用者に近い入口、業務処理、データ管理を分け、どの処理をどの層が担当するかを論理的な役割として整理します。

入口、業務処理、データ管理を分けて考える

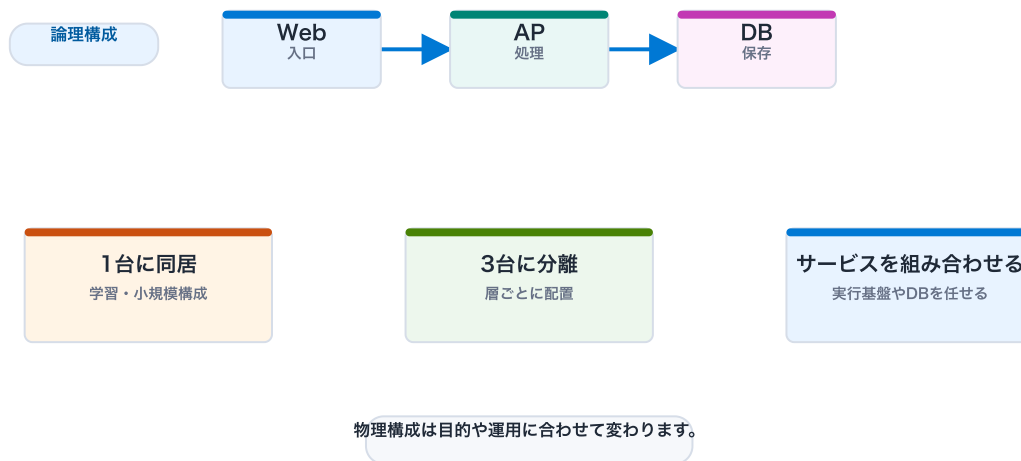


Web 3層は台数ではなく、責務を分けて見るための地図です。

Web 3層は必ず3台ではない

Web 3層はサーバー台数の指定ではなく役割分担の考え方なので、1台同居、複数台分離、マネージドサービス利用のどれでも表現できます。

Web 3層は「3台」ではなく「3つの役割」



処理の流れを追う

ブラウザからの要求が Web 層で受け付けられ、AP 層で処理され、必要に応じて DB 層を参照して結果が返る流れを順番に追います。

要求は入口から処理へ、必要に応じてデータへ進む



処理の主役は要求の種類で変わります。静的配信なら Web 層だけで完結することもあります。

Web 3層のメリットと注意点

層を分けると責任範囲や拡張単位を整理しやすくなりますが、その分だけ通信経路、認証、監視対象が増えることも理解します。

メリット

層ごとに役割を説明しやすい。

Web、AP、DB を別々に拡張しやすい。

担当範囲や変更範囲を分けやすい。

注意点

ネットワークや認証の設計が必要になる。

監視する対象が増える。

障害時に通信経路を追う力が必要になる。

Web 層の役割

Web 層は利用者から最初にアクセスされる入口であり、静的ファイルの配信、HTTP/HTTPS の受付、AP 層への転送などを担当します。

HTTP / HTTPS の要求を受ける。

HTML、CSS、画像などの静的ファイルを返す。

必要に応じて AP 層へ処理を渡す。

Web 層は利用者から見た最初の入口



静的コンテンツと動的コンテンツ

画面や画像のようにそのまま返せる静的コンテンツと、検索結果や Todo 一覧のように処理後に生成される動的コンテンツを区別します。

静的コンテンツ

ファイルをそのまま返す

HTML、CSS、JavaScript、画像など。内容は事前に用意されている。

動的コンテンツ

処理結果を返す

ログイン結果、注文結果、Todo 一覧、API レスポンスなど。AP 層や DB 層と連携する。

Web サーバの代表例

Web サーバには NGINX、Apache HTTP Server、IIS などがあり、第3回では Azure VM 上に NGINX を配置して入口の役割を体験します。

名前	よくある使われ方
NGINX	静的配信、リバースプロキシ、軽量な Web サーバ。
Apache HTTP Server	幅広い環境で使われる汎用 Web サーバ。
IIS	Windows Server と組み合わせて使われる Web サーバ。

第3回で作る Web 層

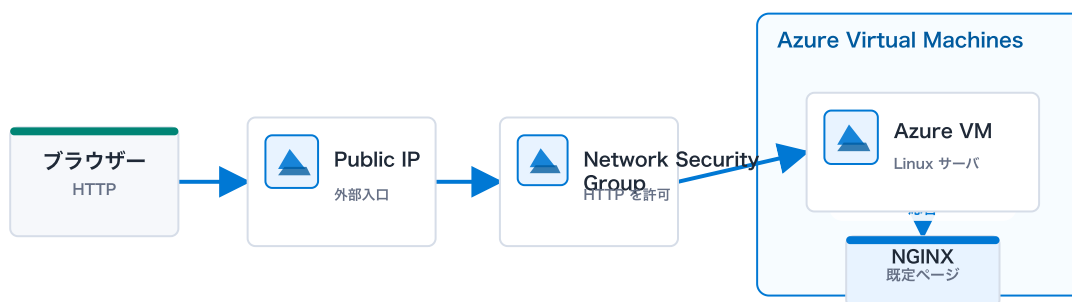
第3回では Public IP、NSG、Azure VM、NGINX をつなげ、インターネットから Web 層に到達できる状態を Azure Portal で確認します。

Public IP から VM にアクセスする。

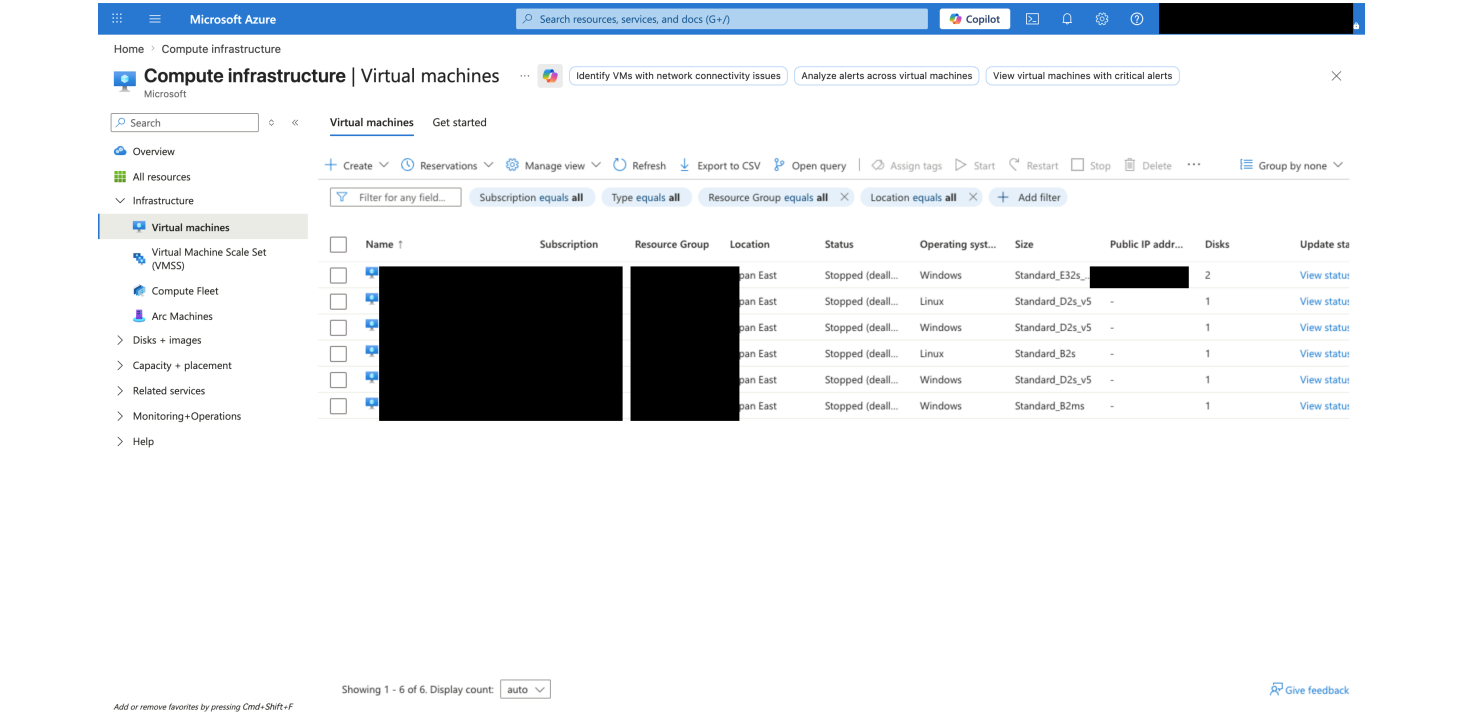
NSG で HTTP 通信を許可する。

NGINX の画面が見えれば入口まで到達できている。

第3回は Web 層の入口までを Azure VM で作る



Web 層だけをまず体験し、入口の通信を確認します。



第3回の予告用参考画像: Azure VM 一覧

AP 層の役割

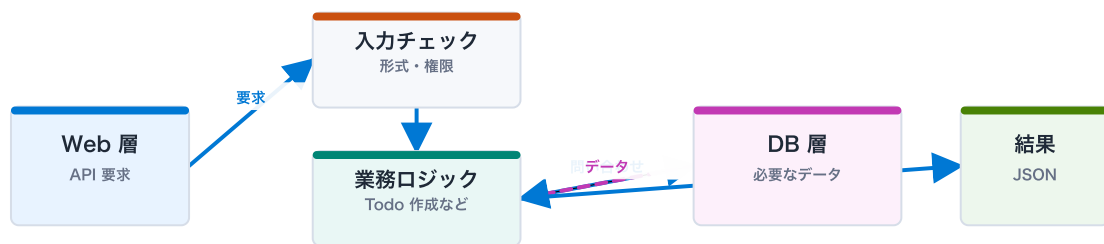
AP 層は利用者の入力や API リクエストを受け取り、入力チェック、業務ルール、DB への問い合わせなどアプリ固有の処理を担当します。

API リクエストを受け取る。

入力チェックや業務ロジックを実行する。

必要なデータを DB 層に問い合わせる。

AP 層はアプリ固有の判断と処理を担当する



AP サーバの代表例

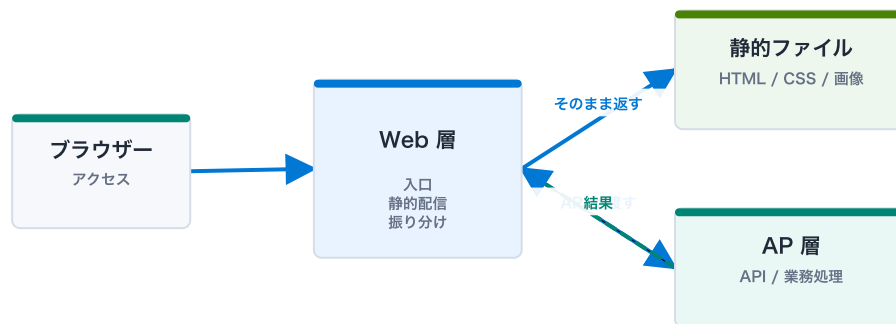
AP 層の実装は Node.js、Java/Spring、.NET、Python/FastAPI など多様で、ここでは製品比較ではなく役割の違いに注目します。

例	位置づけ
Node.js	JavaScript / TypeScript で API を実装する選択肢。
Java / Spring	業務アプリケーションでよく使われる実装基盤。
.NET	Web API や業務処理を構築できる Microsoft 系の基盤。
Python / FastAPI	軽量の API やデータ処理と組み合わせやすい選択肢。

Web 層と AP 層の関係

Web 層と AP 層を分けることで、利用者の入口、静的配信、API 処理、業務ロジックを別々に変更・拡張しやすくなります。

静的配信と動的処理を分ける



DB 層の役割

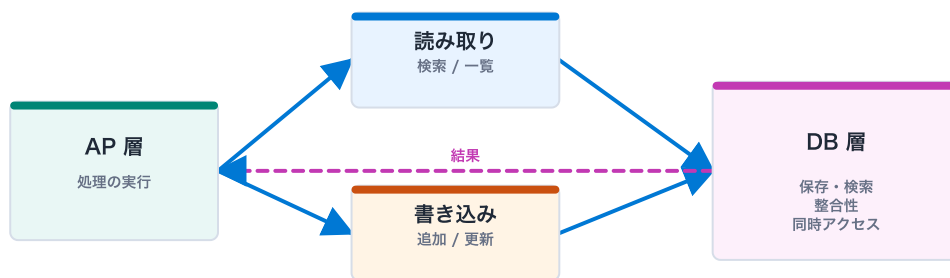
DB 層はアプリケーションのデータを安全に保存し、検索、更新、同時アクセス、トランザクションなどを扱う信頼できるデータ管理場所です。

データの保存、検索、更新を担当する。

同時アクセスや整合性を扱う。

アプリケーションにとって信頼できるデータ管理場所になる。

DB 層はアプリの信頼できるデータ置き場



DB サーバの代表例

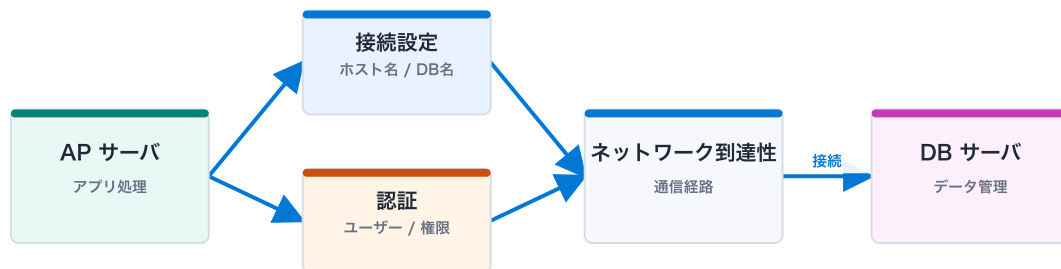
代表的な RDBMS には PostgreSQL、MySQL、SQL Server、Oracle Database があり、第5回から第6回では PostgreSQL を使います。

名前	補足
PostgreSQL	第5回から第6回で Azure Database for PostgreSQL として扱う。
MySQL	Web アプリケーションで広く使われる RDBMS。
SQL Server	Azure SQL Database など Microsoft のマネージド DB とつながる。
Oracle Database	大規模な基幹系システムで見かけることが多い。

AP 層と DB 層の関係

AP 層は自分でデータを保持せず、認証情報、接続設定、ネットワーク到達性を使って DB 層に問い合わせ、必要な結果を受け取ります。

AP 層が DB 層へ接続するには設定と到達性が必要



第5回から第6回で作る DB 層

第5回から第6回では Azure Database for PostgreSQL を使い、Todo アプリのデータ保存を Azure のマネージド DB として扱います。

DB サーバを自分で構築する代わりに、Azure のマネージド DB を使う。

アプリケーションは DB に接続して Todo を保存する。

Todo アプリのデータはマネージド DB に保存する



アプリは DB に接続して利用し、DB 基盤の運用はマネージドサービスに任せます。

Microsoft Azure						
Home						
Azure Database for PostgreSQL flexible servers						
+ Create Manage view Refresh Export to CSV Open query Assign tags Add to service group						
Filter for any field... Subscription equals all Resource Group equals all Location equals all Add filter						
☐	Name ↑	Status	Version	High availability	Location	Resource Group
☐	██████████	Ready	17.9	Disabled	Japan East	██████████

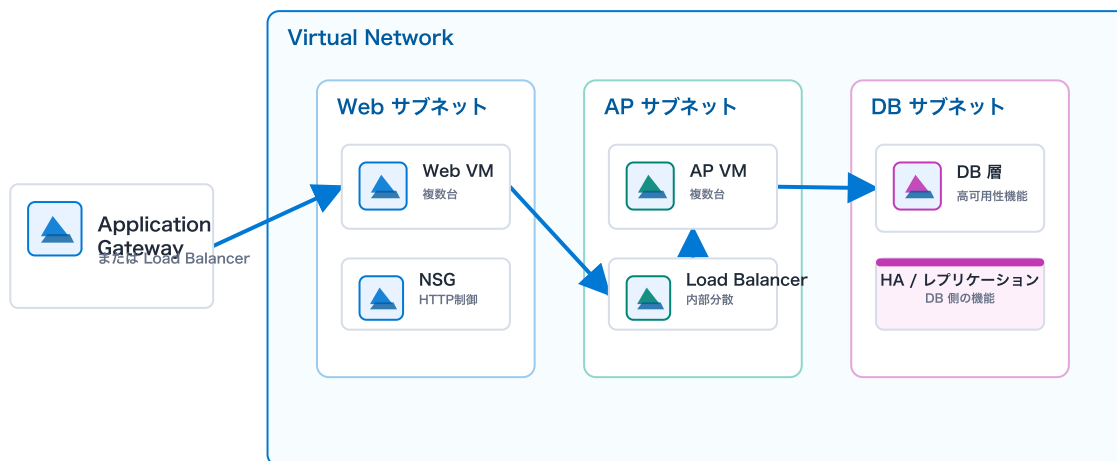
第5回から第6回の予告用参考画像: PostgreSQL 一覧

22 / 32

Azure IaaS で構成する Web 3層

Azure VM を使った IaaS 構成では、Web/AP/DB の各層を分けて配置し、ネットワーク、負荷分散、DB の高可用性を組み合わせます。

IaaS では層ごとの配置と運用範囲を設計する

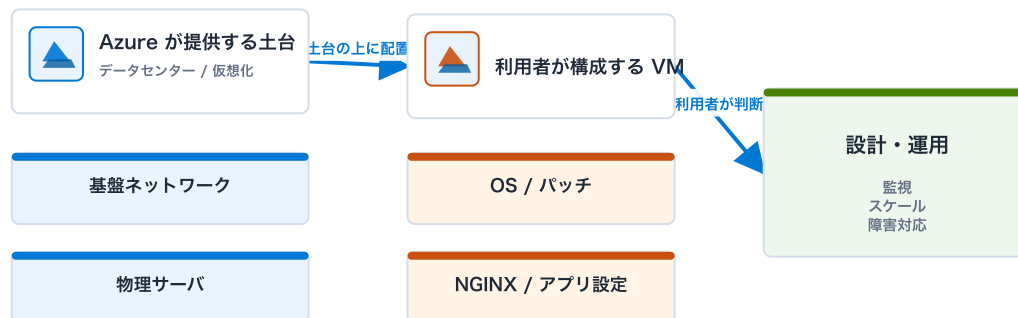


可用性の詳細設計は扱わず、配置と責任範囲の見方だけを確認します。

VM 構成で自分が管理する範囲

VM を使う構成では自由度が高い一方、OS、ミドルウェア、パッチ、監視、スケールなど利用者が設計・運用する範囲も広がります。

VM 構成では自由度と運用責任がセットになる



第3回のハンズオンとの接続

第3回では Web 3層すべてを作るのではなく、まず Web 層に相当する NGINX on Azure VM を作り、入口までの通信を体験します。

第3回ではまず Web 層だけを作る

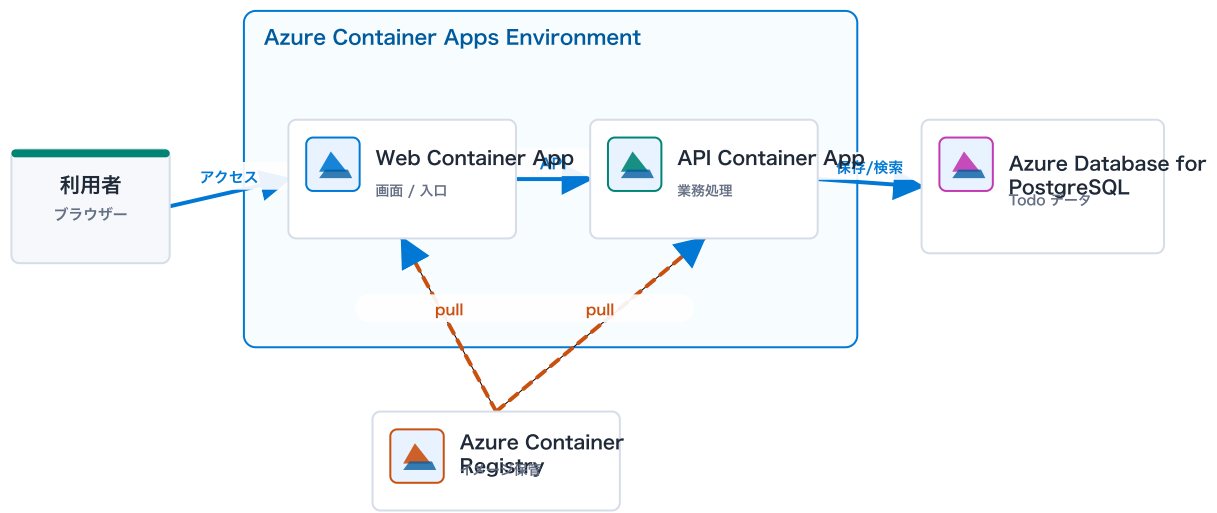


3層すべてを一気に作らず、入口の通信を先に体験します。

Container Apps で構成する Web 3層

Container Apps を使うと、コンテナ化した Web/API をマネージドな実行環境で動かしつつ、DB には PostgreSQL などのサービスを組み合わせられます。

Container Apps では実行基盤を Azure に任せてアプリを動かす



第4回から第6回のつながり

第4回では ACI で単体コンテナを動かし、第5回から第6回では ACA、ACR、PostgreSQL を組み合わせた複数サービス構成に進みます。

第4回: ACI で NGINX コンテナを動かす。

第5回から第6回: ACA、ACR、PostgreSQL で Todo アプリを作る。

単体コンテナから、複数サービスの Todo アプリへ進む



第2回の Web/AP/DB の地図を使って、各回の位置づけを確認します。

Microsoft Azure						
Search resources, services, and docs (G+/)						
Home						
Container Apps						
Locate container apps with errors Get workload templates for container apps List container apps with performance issues						
+ Create Manage view Refresh Export to CSV Open query Assign tags Add to service group Group by none						
Filter for any field... Subscription equals all Resource Group equals all Type equals all Location equals all Add filter						
Name ↑	Type	Resource Group	Location	Subscription	Container Apps Environ...	Environment Type
...	Container App	...	Japan East	...	...	rkload profiles
...	Container App	...	Japan East	...	...	rkload profiles
...	Container App	...	Japan East	...	...	rkload profiles
...	Container App	...	Japan East	...	...	rkload profiles

第5回から第6回の予告用参考画像: Container Apps 一覧

27 / 32

SPA という別の見方

SPA では画面表示の多くをブラウザ側で行い、静的ホスティング、API、データサービスを組み合わせる構成として Web 3層とは別の見方をします。

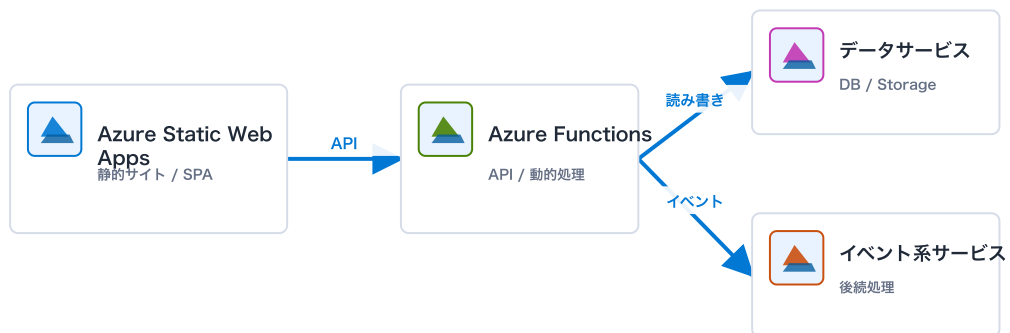
SPA はブラウザ側で画面を組み立てる見方



Static Web Apps で扱うこと

第9回では Azure Static Web Apps を入口に、静的 Web サイト、SPA、Functions/API、データサービスの関係を改めて整理します。

Static Web Apps では静的サイト、API、データを組み合わせる



Web 3層とは別の見方として、第9回で構成の意図を整理します。

RDBMS と NoSQL

データベースはすべて同じではなく、表形式で整合性を重視する RDBMS と、柔軟なデータ構造や分散を得意とする NoSQL を使い分けます。

観点	RDBMS	NoSQL
データの形	表、行、列を中心に整理する。	ドキュメント、キー値、グラフなど多様。
得意なこと	整合性が重要な業務データを扱いやすい。	柔軟なデータ構造や大規模分散に向く。
Azure の例	Azure SQL Database、Azure Database for PostgreSQL	Azure Cosmos DB

今日のまとめ

Web 3層は Azure リソースを理解するための地図であり、次回以降はこの地図の一部を実際の Portal 操作で作って確認します。

Web 3層はリソースを理解するための地図



次回: Web 層を Azure VM で作る

自分の言葉で説明する

クイズでは、入口となる層、3台必須ではないこと、Todo を保存する層を確認し、自分の言葉で第3回へつながる説明にします。

問1

利用者からのリクエストを最初に受ける入口に相当する層はどれですか。

正解: Web 層

問2

Web 3層アーキテクチャは、必ず3台のサーバで構成しなければならない。正しいですか。

正解: 正しくない

問3

Todo アプリで「Todo を保存する」役割にもっとも近い層はどれですか。

正解: DB 層